

* Post's Correspondence Problem (PCP) $\rightarrow$ undecidable:

$\hookrightarrow$ small instances of the problem might have solⁿ

- PCP is a famous undecidable problem in computation introduced by Emil Post in 1946.
- Is used to prove undecidability of many other problems using reduction.
eg: PCP can be used to prove:
 - (i) CFG ambiguity is undecidable
 - (ii) Equivalence of CFG is undecidable
 - (iii) Intersection of 2 CFG is undecidable
- PCP deals with matching strings formed by concatenation under certain rules

domino
 $\downarrow$

Definition: A finite set of dominos (pair of strings - binary or characters)

$$P = \{ (x_1, y_1), (x_2, y_2), \dots, (x_n, y_n) \} \text{ where,}$$

$x_i, y_i \in \Sigma^*$, Σ is a finite alphabet, $n \geq 1$

the decision problem is, determine whether there exists a non-empty sequence of indices $i_1, i_2, \dots, i_k$ ($k \geq 1$) such that $\overset{\text{Top}}{x_{i_1} x_{i_2} \dots x_{i_k}} = \overset{\text{Bottom}}{y_{i_1} y_{i_2} \dots y_{i_k}}$ where $1 \leq i_j \leq n$

Note: Steps to solve:

- ① PickUp any domino
- ② Reuse dominos
- ③ Arrange dominos in any order
- ④ Concat top strings and concat bottom strings separately
- ⑤ Goal is to make concatenated strings identical

→ Modified PCP:

- In MPCP the sequence must start with the first domino only

Definition: Given: $P = \{ (x_1, y_1), (x_2, y_2), \dots, (x_n, y_n) \}$

we need to find a sequence: $i_1, i_2, i_3, \dots, i_k$
such that $\underbrace{x_{i_1} x_{i_2} x_{i_3} \dots x_{i_k}}_{\text{fixed}} = \underbrace{y_{i_1} y_{i_2} y_{i_3} \dots y_{i_k}}_{\text{fixed}}$

Note: • PCP is undecidable

• there is no algo to solve PCP

• there is no TM that can decide all instances of PCP

→ Prove the PCP is undecidable.

$$\star \text{Accept}_{\text{TM}} \leq \text{MPCP} \leq \text{PCP}$$

Proof: using reduction

(i) start with a known undecidable problem, that is acceptance problem of Turing machine

- (ii) Reduce it to MPCP
- (iii) Reduce MPCP to PCP
- (iv) Conclude that PCP is undecidable

$$\star \text{Accept}_{\text{TM}} \leq \text{MPCP} \leq \text{PCP}$$

since $\text{accept}_{\text{TM}}$ is undecidable, PCP is undecidable

$\star$ Bigger Q,
Less
Marks
 $\Downarrow$
No solution

Q. Check whether following PCP has soln

$$P = \{(a, ab), (ba, a), (b, bb)\}$$

Soln

a	ba	b
ab	a	bb

(1,2)

Top: $a \cdot ba = aba$

Bottom: $ab \cdot a = aba$

(3,2)

top : bba

bottom: bba

Note: • PCP solution if it exists we may eventually find it
• if no solution exists then, algorithm may run forever

↳ Recursively Enumerable by Not decidable

Note: pair + concatenate + problem

Q. Solve the given PCP instance

i	x_i	y_i
1	110	110110
2	0011	00
3	0110	110

Solⁿ No solution

Q. Solve the given instance of PCP

i	X _i	Y _i
1	1	010
2	0	10
3	10	101
4	01	100

* find common prefix.

Soln No soln

Q.	i	X _i	Y _i
	1	11	111
	2	100	001
	3	111	11

Soln (1, 3), (3, 1)

Q.	i	X _i	Y _i
	1	1	10
	2	0	10
	3	010	01
	4	11	1

Soln (4, 2)
 top : 110
 bottom : 110

Q.	i	x_i	y_i
	1	10	101
	2	01	100
	3	0	10
	4	100	0
	5	1	010

Solⁿ No solⁿ

Q.	i	x_i	y
	1	1	11
	2	01	1
	3	10	010

Solⁿ No solⁿ

Q.	i	x_i	y_i
	1	0	1
	2	01	10
	3	10	01

Soln No soln

Q .	i	x_i	y_i
	1	10	101
	2	011	11
	3	101	011

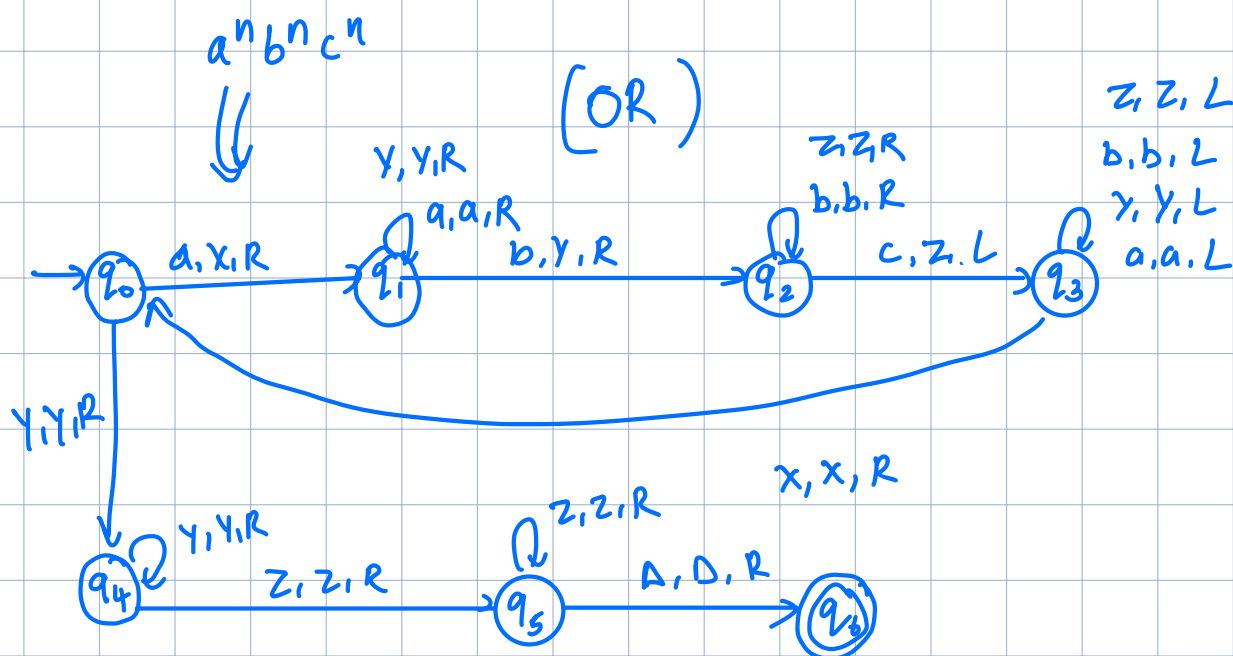
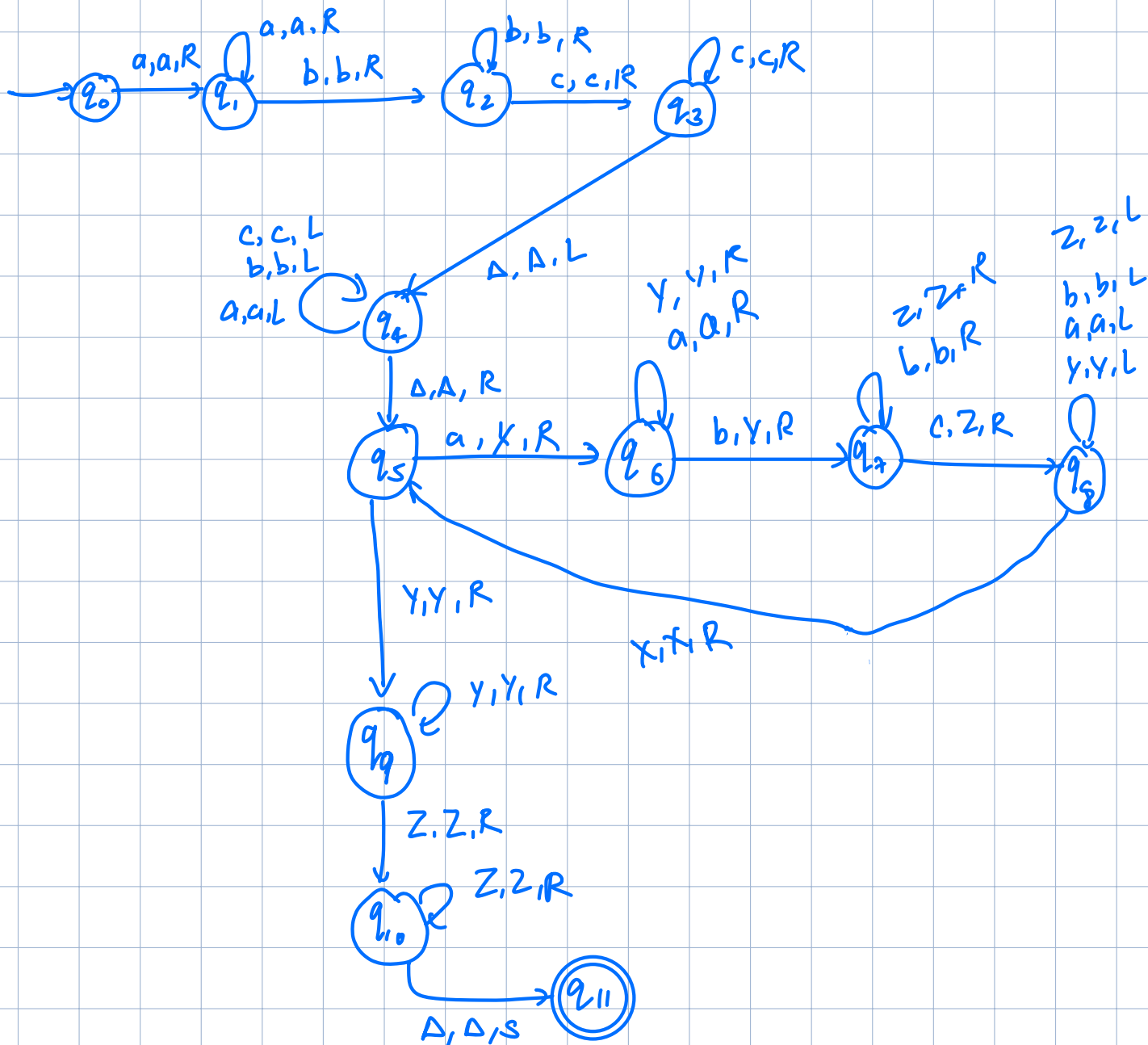
Soln No soln

Q .	i	x_i	y_i
	1	110	11
	2	0	011
	3	11	0

Soln

Q. $a^*b^*c^*$ Draw TM and time complexity.
($a^n b^n c^n, n \geq 0$)

Solⁿ



8.1 | RECURSIVELY ENUMERABLE AND RECURSIVE

Definition 8.1 Accepting a Language and Deciding a Language

A Turing machine T with input alphabet Σ accepts a language $L \subseteq \Sigma^*$ if $L(T) = L$. T decides L if T computes the characteristic function $\chi_L : \Sigma^* \rightarrow \{0, 1\}$. A language L is *recursively enumerable* if there is a TM that accepts L , and L is *recursive* if there is a TM that decides L .

[THEOREM]: Every recursive language is recursively enumerable
proof:

Suppose T is a TM that decides $L \subseteq \Sigma^*$. An algorithm to accept L is the following: Given $x \in \Sigma^*$, execute T , on input x . T will halt and produce an output; if the output is 1 (i.e. if $\chi_L(x) = 1$), accept, and if output is 0, reject.

[THEOREM]: IF $L \subseteq \Sigma^*$ is accepted by a TM T that halts on every input string, then L is recursive.

Proof:

If T halts on every input string, then the following is an algorithm for deciding L : Given $x \in \Sigma^*$, if T accepts x , return 1, and if T rejects x , return 0.

[THEOREM]: If L_1 and L_2 are both recursively enumerable languages over Σ , then $L_1 \cup L_2$ and $L_1 \cap L_2$ are also recursively enumerable

[THEOREM]: If L_1 and L_2 are both recursive languages over Σ , then $L_1 \cup L_2$ and $L_1 \cap L_2$ are also recursive

[THEOREM]: If L is a recursive language over Σ , then its complement L' is also recursive.

[THEOREM]: If L is a recursively enumerable language, and its complement L' is also recursively enumerable, then L is recursive and so is L'

Note: Accepting is weaker than deciding

Note: Recursive $\subseteq$ Recursively Enumerable.

Definition 8.8 A TM Enumerating a Language

Let T be a k -tape Turing machine for some $k \geq 1$, and let $L \subseteq \Sigma^*$. We say T enumerates L if it operates such that the following conditions are satisfied.

1. The tape head on the first tape never moves to the left, and no nonblank symbol printed on tape 1 is subsequently modified or erased.
2. For every $x \in L$, there is some point during the operation of T when tape 1 has contents

$$x_1 \# x_2 \# \dots \# x_n \# x \#$$

for some $n \geq 0$, where the strings $x_1, x_2, \dots, x_n$ are also elements of L and $x_1, x_2, \dots, x_n, x$ are all distinct. If L is finite, then nothing is printed after the $\#$ following the last element of L .

CHARACTERIZATION THEOREM

Theorem 8.9

For every language $L \subseteq \Sigma^*$, L is recursively enumerable if and only if there is a TM enumerating L , and L is recursive if and only if there is a TM that enumerates the strings in L in canonical order (see Section 1.4).

Proof

We have to prove, for an alphabet Σ and a language $L \subseteq \Sigma^*$, these four things:

1. If there is a TM that accepts L , then there is a TM that enumerates L .
2. If there is a TM that enumerates L , then there is a TM that accepts L .
3. If there is a TM that decides L , then there is a TM that enumerates L in canonical order.
4. If there is a TM that enumerates L in canonical order, then there is a TM that decides L .

Theorem 8.9 — CHARACTERIZATION THEOREM

This theorem connects everything.

Part 1

Acceptable $\Rightarrow$ Enumerable

If a TM accepts L , then there exists a TM that enumerates L .

Idea

- Try all strings
- Simulate acceptance **step by step**
- Print when accepted

Uses **dovetailing**

Dovetailing Explained

Canonical order:

$\epsilon, a, b, aa, ab, ba, bb, \dots$

Passes:

- Pass 1: 1 step on ϵ
- Pass 2: 2 steps on ϵ , 1 on a
- Pass 3: 3 on ϵ , 2 on a , 1 on b
- ...

✓ Guarantees:

- Any string accepted in k steps will be found



Part 2

Enumerable $\Rightarrow$ Acceptable

If a TM enumerates $\bar{L}$:

- To accept x , just watch the enumeration
- If x appears $\rightarrow$ accept
- Otherwise $\rightarrow$ may loop forever

✓ Accepts L

Part 3

Decidable $\Rightarrow$ Canonical Enumeration

If T decides L :

- Check each string in canonical order
- Include only those accepted

✓ Order preserved

✓ Complete

Part 4

Canonical Enumeration $\Rightarrow$ Decidable

This is subtle.

Given input x :

- Watch enumeration
- If x appears $\rightarrow$ accept
- If a later string appears first $\rightarrow$ reject

Why?

- Canonical order ensures correctness



Special Case: Finite Languages

Enumeration may stop silently.

But:

- Every finite language is regular
 - Hence recursive
 - So decidability still holds
-

FINAL CONSOLIDATED TABLE (MEMORIZE)

Property	Accept	Decide	Enumerate
Always halts	✗	✓	✗
Membership certainty	Partial	Complete	Partial
Complement closed	✗	✓	✗
Equivalent power	RE	Recursive	RE

One-sentence takeaway

Recursive = enumerable + complement enumerable

ONE-LINE MEMORY HOOKS

- Recursive = always answers
- RE = answers YES only
- RE + complement RE = recursive
- Enumerate = accept
- Canonical enumerate = decide

→ most powerful.

⇒ Unrestricted Grammar: ⇒ correspond to RE languages

An unrestricted grammar is a 4-tuple $G = (V, \Sigma, S, P)$, where V and Σ are disjoint sets of variables and terminals, respectively. S is an element of V → start symbol, and P is a set of productions of the form

$$\boxed{\alpha \rightarrow \beta}$$

where, $\alpha, \beta \in (V \cup \Sigma)^*$ and α consists at least one variable.

Note: A language is recursively enumerable if and only if it is generated by an unrestricted grammar

Note: Unrestricted grammar can loop forever.

★
Q. Define multitape Turing machine. Explain how multitape TM can be simulated using single tape TM.

Definition

A **multitape Turing machine** is a Turing machine that has **multiple tapes**, each with its own **read/write head**.

In one transition, the machine can read symbols from all tapes, write symbols on all tapes, and move each head independently.

Formal Definition

A **k-tape Turing Machine** is a 7-tuple:

$$M = (Q, \Sigma, \Gamma, \delta, q_0, B, F)$$

where:

- Q : finite set of states
- Σ : input alphabet
- Γ : tape alphabet ($\Sigma \subseteq \Gamma$)
- B : blank symbol
- q_0 : initial state
- F : set of accepting states
- δ : transition function

$$\delta : Q \times \Gamma^k \rightarrow Q \times \Gamma^k \times \{L, R, S\}^k$$

Working of a Multitape Turing Machine

- Input is placed on **tape 1**
- Other tapes are initially **blank**
- Each step reads one symbol from each tape
- Each tape head moves independently

Simulation of a Multitape TM by a Single-Tape TM

Statement

Every multitape Turing machine can be simulated by an equivalent single-tape Turing machine. Thus, multitape TMs do not increase computational power, only efficiency.

Idea of Simulation

1. Tape Encoding

- All k tapes are encoded on **one tape**
- Tape contents are separated using a special symbol (e.g., **#**)
- Example format:

```
shell
# tape1 # tape2 # tape3 #
```

Copy code

2. Head Position Marking

- The position of each tape head is marked using a **special symbol** or notation (e.g., underlining or tagged symbols)

3. Simulation of One Step

For each transition of the multitape TM:

- The single-tape TM scans the tape to locate all head positions
- Reads the symbols under each simulated head
- Updates symbols and head positions accordingly
- Returns to the beginning for the next step

4. Movement Simulation

- Left/Right movements are simulated by shifting head markers
- Requires multiple scans of the tape



[THEOREM]: For every unrestricted grammar G , there is a Turing machine T with $L(T) = L(G)$

[THEOREM]: For every Turing Machine T with input alphabet Σ , there is an unrestricted grammar generating the language $L(T) \subseteq \Sigma^*$

⇒ Context Sensitive Grammar:

Definition 8.16 Context-Sensitive Grammars

A *context-sensitive grammar* (CSG) is an unrestricted grammar in which no production is length-decreasing. In other words, every production is of the form $\alpha \rightarrow \beta$, where $|\beta| \geq |\alpha|$.

A language is a *context-sensitive language* (CSL) if it can be generated by a context-sensitive grammar.

Note: Because a context-sensitive grammar cannot have ϵ -productions, no language containing ϵ can be a CSL.

Note: Every Context-free language is either a CSL or the union of $\{\epsilon\}$ and CSL.

⇒ Linear Bounded Automata:

[DEFINITION]: A linear-bounded automaton (LBA) is a 5-tuple $M = (Q, \Sigma, \Gamma, q_0, \delta)$ that is identical to a nondeterministic Turing machine, with the following exception. There are 2 extra tape symbols, $[$ and $]$, assumed not to be elements of the tape alphabet Γ . The initial configuration of M corresponding to input x is $q_0 [x]$, with the symbol $[$ in the leftmost square and the symbol $]$ in the first square to the right of x .

During its computation, M is not permitted to replace either of these brackets or to move its tape head to the left of the $[$ or to the right of the $]$.

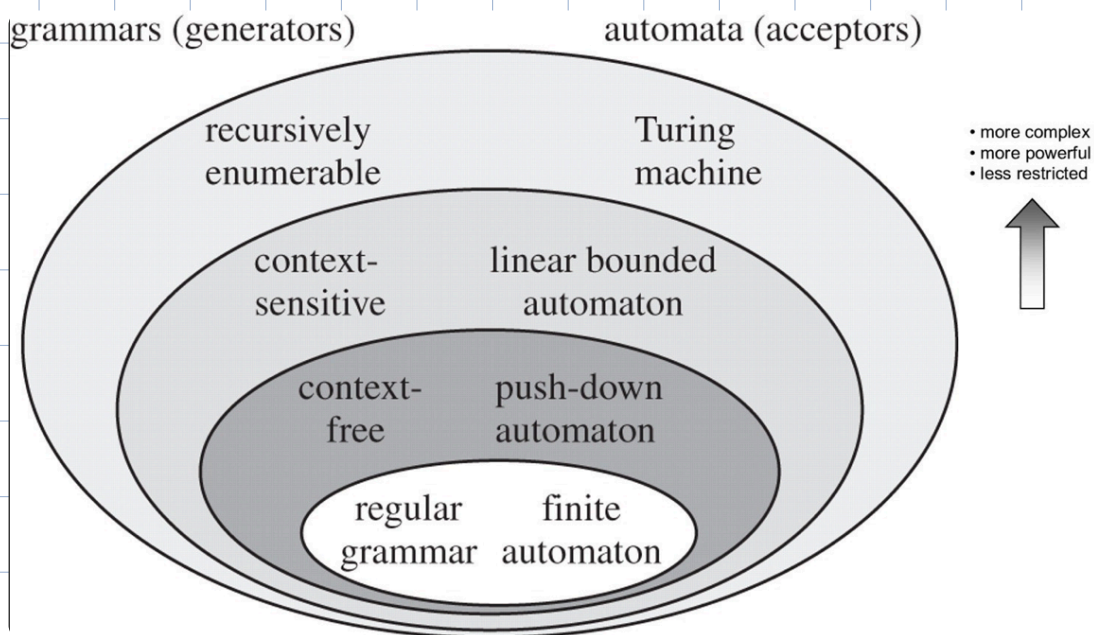
[THEOREM]: If $L \subseteq \Sigma^*$ is a context-sensitive language, then there is a linear-bounded automaton that accepts L .

[THEOREM]: If $L \subseteq \Sigma^*$ is accepted by a LBA $M = (Q, \Sigma, \Gamma, q_0, \delta)$, then there is a context-sensitive grammar G generating $L - \{\epsilon\}$

⇒ CHOMSKY HIERARCHY:

Table 8.21 | The Chomsky Hierarchy

Type	Languages (Grammars)	Form of Productions in Grammar	Accepting Device
3	Regular	$A \rightarrow aB, A \rightarrow \Lambda$ ($A, B \in V, a \in \Sigma$)	Finite automaton
2	Context-free	$A \rightarrow \alpha$ ($A \in V, \alpha \in (V \cup \Sigma)^*$)	Pushdown automaton
1	Context-sensitive	$\alpha \rightarrow \beta$ ($\alpha, \beta \in (V \cup \Sigma)^*, \beta \geq \alpha $, α contains a variable)	Linear-bounded automaton
0	Recursively enumerable (unrestricted)	$\alpha \rightarrow \beta$ ($\alpha, \beta \in (V \cup \Sigma)^*$, α contains a variable)	Turing machine



[THEOREM]: Every context - sensitive language L is recursive.

Note: $S_3 \subseteq S_2 \subseteq S_1 \subseteq R \subseteq S_0$

where R is the set of recursive language, S_i is the set of languages of type i for $i=0$ and $i=1$, and for $i=2$ and $i=3$, S_i is the set of languages of type i that do not contain ϵ

$\Rightarrow$ Not Every Language is Recursively Enumerable:

[DEFINITION: A set A of the same size as B or larger than B]:

2 sets A and B , either finite or ∞ , are the same size if there is a bijection $f: A \rightarrow B$. A is larger than B if some subset of A is the same size as B but A itself is not.

[DEFINITION: Countably infinite and countable sets]: A set A is countably infinite (the same size of $\mathbb{N}$ (set of natural nos)) if there is a bijection $f: \mathbb{N} \rightarrow A$, or a list $a_0, a_1, \dots$ of elements of A such that every element of A appears exactly once in the list. A is countable if A is either finite or countably infinite.

Theorem 8.25

Every infinite set has a countably infinite subset, and every subset of a countable set is countable.

Proof

The first statement was discussed above. If A is countable and $B \subseteq A$, it is sufficient to consider the case when A and B are both infinite. In this case, $A = \{a_0, a_1, a_2, \dots\}$, and the set $I = \{i \in \mathcal{N} \mid a_i \in B\}$ is infinite. If we let j_0 be the smallest number in I , j_1 the next smallest, and so on, then $B = \{b_{j_0}, b_{j_1}, \dots\}$, so that B is countable.

Note: • A Countable Union of Countable Sets is Countable

- Languages are Countable Sets
- The set of Turing machines is countable
- The set of $2^{\mathcal{N}}$ is Uncountable.

[THEOREM]: Not all languages are recursively enumerable. In fact, the set of languages over $\{0,1\}$ that are not recursively enumerable is uncountable

Proof

In Example 8.31 we showed that the set of subsets of $\mathcal{N}$ is uncountable, and we observed that because $\{0,1\}^*$ is the same size as $\mathcal{N}$, it follows that the set of languages over $\{0,1\}$ is uncountable. According to Example 8.30, the set of recursively enumerable languages over $\{0,1\}$ is countable. The theorem follows from the fact that if T is any countable subset of an uncountable set S , $S - T$ is uncountable (see Exercise 8.38).

⇒ Decidability:

As it turns out, not only can we describe a language that is not recursively enumerable, but we can do it by using the same diagonal argument that we used for the uncountability result. We showed that for every list $A_0, A_1, \dots$ of subsets of $\mathcal{N}$, there is a subset A of $\mathcal{N}$ that is not in the list. The fact that the sets A_i were in a *list* was not crucial; here are the crucial steps.

1. We started with a collection of subsets A_i of $\mathcal{N}$, each one associated with a specific *element* of $\mathcal{N}$ (namely, i).
2. We defined another subset A , containing the elements i of $\mathcal{N}$ that do not belong to the subset A_i associated with i .

The conclusion was that for each i , $A \neq A_i$, because $i \in A \Leftrightarrow i \notin A_i$.

Now we want to find a language $L \subseteq \{0, 1\}^*$ that cannot be accepted by a Turing machine—in other words, a language that is different from $L(T)$, for every Turing machine T with input alphabet $\{0, 1\}$. The string $e(T)$ introduced in Section 7.8 is “associated with” $L(T)$, just as i is associated with A_i , and we can repeat the argument as follows:

1. We have a collection of subsets $L(T)$ of $\{0, 1\}^*$, each one associated with a specific element of $\{0, 1\}^*$ (namely, $e(T)$).
2. We define another subset L , containing the elements $e(T)$ of $\{0, 1\}^*$ that do not belong to the subset $L(T)$ associated with $e(T)$.

The conclusion this time is that for each T , $L \neq L(T)$, because $e(T) \in L \Leftrightarrow e(T) \notin L(T)$.

Definition 9.1 The Languages NSA and SA

Let

$$NSA = \{e(T) \mid T \text{ is a TM, and } e(T) \notin L(T)\}$$

$$SA = \{e(T) \mid T \text{ is a TM, and } e(T) \in L(T)\}$$

(NSA and SA stand for “non-self-accepting” and “self-accepting.”)

[THEOREM]: The language NSA is not recursively enumerable. The language SA is recursively enumerable but not recursive

[DEFINITION: Decidable Problems]: If P is a decision problem, and e is a reasonable encoding of instances of P over the alphabet Σ , we say that P is decidable if $\gamma(P) = \{e(I) \mid I \text{ is a yes-instance of } P\}$ is a recursive language

[THEOREM]: The decision problem Self-Accepting is undecidable

[THEOREM]: For every decision problem P , P is decidable if and only if this complementary problem P' is decidable.

1 Big Picture: What is this chapter trying to show?

This chapter is about limits of computation.

Are there well-defined questions that no algorithm can always answer correctly?

Surprisingly:

- ✓ The question can be clear
- ✓ The rules can be precise
- ✗ But no computer program can solve it for all cases

That's what undecidable means.

6 Two important languages: SA and NSA

Let's define them very plainly.

✓ SA (Self-Accepting)

ini

Copy code

$SA = \{ e(T) \mid T \text{ accepts its own encoding} \}$

These are machines that say "YES" to themselves.

✗ NSA (Non-Self-Accepting)

ini

Copy code

$NSA = \{ e(T) \mid T \text{ does NOT accept its own encoding} \}$

These machines either:

- reject their own encoding, or
- loop forever on it

⇒ Unsolvable Problems:

An unsolvable problem is a problem for which no algorithm can always give the correct answer for every input, even if the problem is clearly defined, the input is finite and the rules are precise.

Note: A decision problem P is unsolvable (undecidable) if: there is no Turing machine that decides P . (ie. halts and answers correctly on all inputs)

Key comparison table

Type of problem	Meaning
Decidable	Algorithm always halts and answers correctly
Semi-decidable	Algorithm halts only on YES cases
Unsolvable / Undecidable	No algorithm can solve all cases

Examples of unsolvable problems

1 Self-Accepting Problem

Given a TM T , does T accept $e(T)$?

✗ Unsolvable

2 Halting Problem (most famous)

Given a program P and input x , will P halt?

✗ Unsolvable

3 Program equivalence

Do two programs always produce the same output?

✗ Unsolvable

⇒ Reducing One Problem to another:

- Reduction is a way to solve one problem by converting it into another problem.

Definition 9.6 Reducing One Decision Problem to Another, and Reducing One Language to Another

Suppose P_1 and P_2 are decision problems. We say P_1 is reducible to P_2 ($P_1 \leq P_2$) if there is an algorithm that finds, for an arbitrary instance I of P_1 , an instance $F(I)$ of P_2 , such that for every I , the answers for the two instances are the same, or I is a yes-instance of P_1 if and only if $F(I)$ is a yes-instance of P_2 .

If L_1 and L_2 are languages over alphabets Σ_1 and Σ_2 , respectively, we say L_1 is reducible to L_2 ($L_1 \leq L_2$) if there is a Turing-computable function $f : \Sigma_1^* \rightarrow \Sigma_2^*$ such that for every $x \in \Sigma_1^*$,

$$x \in L_1 \text{ if and only if } f(x) \in L_2$$

Theorem 9.7

Suppose $L_1 \subseteq \Sigma_1^*$, $L_2 \subseteq \Sigma_2^*$, and $L_1 \leq L_2$. If L_2 is recursive, then L_1 is recursive.

Suppose P_1 and P_2 are decision problems, and $P_1 \leq P_2$. If P_2 is decidable, then P_1 is decidable.

Proof

To prove the first statement, let $f : \Sigma_1^* \rightarrow \Sigma_2^*$ be a function that reduces L_1 to L_2 , and let T_f be a TM computing f . Let T_2 be another TM that decides L_2 . Let T be the composite TM $T_f T_2$. On input x , T first computes $f(x)$ and then decides, by returning output 1 or 0, whether $f(x) \in L_2$. Because f is a reduction, this is the same as deciding whether $x \in L_1$. Therefore, T decides the language L_1 .

Suppose now that F is an algorithm reducing P_1 to P_2 , and that we have encoding functions e_1 and e_2 for the instances of P_1 and P_2 , respectively. Our assumption is that $Y(P_2)$, the subset of Σ_2^* containing the strings that represent yes-instances of P_2 , is recursive, and we wish to show that $Y(P_1)$ is also recursive.

For an arbitrary string $x \in \Sigma_1^*$, we can decide, assuming that e_1 is a reasonable encoding, whether x represents an instance of P_1 . If not, then $x \notin Y(P_1)$. If $x \in Y(P_1)$, then x represents an instance I of P_1 , and we have this sequence of equivalences:

$$\begin{aligned} x = e_1(I) \in Y(P_1) &\Leftrightarrow I \text{ is a yes-instance of } P_1 \\ &\Leftrightarrow F(I) \text{ is a yes-instance of } P_2 \\ &\Leftrightarrow e_2(F(I)) \in Y(P_2) \end{aligned}$$

The statement in the second line is true because F is a reduction from P_1 to P_2 . Because $Y(P_2)$ is recursive, we have an algorithm to decide whether $e_2(F(I)) \in Y(P_2)$, and so we have an algorithm to decide whether $x \in Y(P_1)$.

[THEOREM]: Both Accepts and Halts are undecidable

[THEOREM]: The following five decision problems are undecidable:

- ① Accepts- λ : Given a TM T , is $\lambda \in L(T)$?
- ② Accepts Everything: Given a TM T with input alphabet Σ , is $L(T) = \Sigma^*$?
- ③ Subset: Given 2 TMs T_1 and T_2 is $L(T_1) \subseteq L(T_2)$?
- ④ Equivalent: Given 2 TMs T_1 and T_2 , is $L(T_1) = L(T_2)$?
- ⑤ Write Symbol: Given a TM T and a symbol a in the tape alphabet of T does T ever write an a if it starts with an empty tape?

Definition 9.11 A Language Property of TMs

A property R of Turing machines is called a *language property* if, for every Turing machine having property R , and every other TM T_1 with $L(T_1) = L(T)$, T_1 also has property R . A language property of TMs is *nontrivial* if there is at least one TM that has the property and at least one that doesn't.

⇒ The Halting Problem:

- The halting problem is the problem of determining, for a given program and input, whether the program will eventually halt or run forever.

let M be a TM and w be an input string

$$\text{HALT} = \{ \langle M, w \rangle \mid M \text{ halts on input } w \}$$

The Halting problem asks whether there exists a TM H such that:

$$H(\langle M, w \rangle) = \begin{cases} \text{YES, if } M \text{ halts on } w \\ \text{No, if } M \text{ does not halt on } w \end{cases}$$

and H halts for all inputs

The Halting Problem is undecidable.

There exists no Turing machine that can correctly decide for every program - input pair whether the program halts.

Proof Sketch (By Contradiction)

1. Assume there exists a Turing Machine H that decides HALT.
2. Construct a machine D that:
 - Takes $\langle M \rangle$ as input
 - Runs $H(\langle M, M \rangle)$
 - Loops forever if H says "halts"
 - Halts if H says "loops"
3. Run D on itself: $D(\langle D \rangle)$
4. This leads to a contradiction in both cases.
5. Hence, such a machine H cannot exist.

∴ HALT is undecidable

Significance

- Shows a fundamental limitation of computation.
- Proves that no algorithm can detect infinite loops in all programs.
- Forms the basis for many undecidability results (e.g., Rice's Theorem).

Conclusion

The Halting Problem demonstrates that there exist well-defined computational problems that cannot be solved algorithmically, regardless of time or hardware.

⇒ Time and Space Complexity of TMs:

(1) Time Complexity:

The time complexity of a TM is the number of steps (moves) the TM makes before halting, as a function of the input size n .

Formally, for a TM M :

$$T_M(n) = \max \{ \text{no. of steps taken by } M \text{ on any input of length } n \}$$

- each read, write, or head movement counts as one step
- worst-case time is considered.

T-C Classes :

- Linear time: $O(n)$
- Polynomial time: $O(n^k)$
- Exponential time: $O(2^n)$

Note: A language is in P if it can be decided by a deterministic TM in polynomial time

(2) Space Complexity:

The space complexity of a TM is the max no. of tape cells used during computation, as a function of input size n .

$$S_M(n) = \max \{ \text{no. of cells visited in } M \text{ on any input of length } n \}$$

- includes input tape cells and work tape cells
- blank cells not visited are not counted

S-C classes:

- Constant space: $O(1)$
- Logarithmic space: $O(\log n)$
- Polynomial space: $O(n^k)$

Note: A language is in PSPACE if it can be decided using polynomial space.

Note: ▷ A TM may use large time but small space.

2> Space is reusable, but time is not

3> Every computation taking $T(n)$ time uses at most $T(n)$ space
 $S(n) \leq T(n)$